

Explanation of Markov Chain ‘Weasel’ Program

Aditya Savanur

September 19, 2026

I first encountered this problem in high school, it’s called Dawkins’ weasel. It’s a simple qualitative model of evolution (technically, artificial selection) which is meant to illustrate how selection and mutation on a gene sequence is far faster than blind random chance. Technically, this stretches the definition of qualitative model a little bit (qualitative models have to approximately reproduce the data; this only reproduces the single fact that random chance is much slower than selection + mutation), but biologists will be biologists. This is in fact a simple example of an evolutionary algorithm, specifically a (1, N) evolutionary strategy¹.

In high school, I coded this up as a simple exercise for my own understanding, then I searched for mathematical analyses on the net but failed to find any (I did not know yet about evolutionary algorithms). The closest I came was a blog by a guy calling himself ‘Dieben’², which gives some graphs and some R code with German variable names but almost no explanations of how he got his results. I tried and failed to solve it myself, gave up on it for a while, and then picked it back up once my skills had increased.

The basic setup is as follows: You want to reproduce a given text string of length L , the target (Dawkins used the Shakespeare quote “Methinks it is like a weasel”). The alphabet that the letters are drawn from has A letters. Now, start with a random string. Generate N copies (reproduction). In each resulting “offspring”, “mutate” each letter with a given probability μ . Select the offspring that is closest to the target, distance being measured by the number of incorrect letters (Hamming distance), and repeat the preceding steps with the offspring in the place of the original random string. Keep doing this until you reach the target. The mathematical question is: what is the distribution of the number of steps it takes to reach the target? What is the analogous distribution for completely random mutation without selection? We can do some estimates of the mean and then proceed to solve the full problem in steps.

Let’s start with the second question first, as it is far easier. Assume that each letter mutates independently, then this is a geometric distribution by definition. Since the alphabet has A letters, the probability of choosing the correct letter by chance is $1/A$. The probability that all L characters are correct is thus $(1/A)^L$. Since this is a geometric distribution, the mean number of steps to success is simply A^L , or about 10^{40} steps for English letters.

We can now estimate the mean number of steps under mutation and selection as follows: the initial number of correct letters is L/A , since each character has a $1/A$ probability of being correct. The final number of correct letters must be L . The average number of characters that mutate per step is $\mu \times L$, and assuming all of these are correct [this is the selection assumption, we will see corrections to it later], we can solve the resulting equation

$$\frac{L}{A} + \mu \times L \times k = L$$

to get $\frac{1-1/A}{\mu}$ for the expected number of steps k . For the same parameters as the previous setup, and a mutation probability of 0.05, we get about 20 steps. At least in the parameter regime where this approximation holds, this is already enough to show that artificial selection is astronomically faster than random chance, and that the algorithm runtime is constant considered as a function of L , but we must go further. (The implied number of correct characters per step is insanely small for random chance, of course.)

¹<https://direct.mit.edu/evco/article-abstract/1/1/1/1092/An-Overview-of-Evolutionary-Algorithms-for>

²<https://dieben.blogspot.com/2009/08/dawkins-weasel-it-isnt-that-bad.html>

We can now do the simplest case of not-completely-random evolution, and will build up to the general case. Assume we have a single letter that we want to evolve ($L = 1$). The state of the letter does not depend on the history of what happened before, it just depends on its current state, so this is a Markov chain. Since there are two states, correct and incorrect, the transition matrix is 2x2. The first entry in the transition matrix (the transition probability between the states 'incorrect' and 'incorrect') can be found as follows. There are two possibilities for what could happen: that the character does not mutate (probability $1 - \mu$) or that it mutates and the resulting character is incorrect (probability $\mu(1 - 1/A)$). These are the only two possibilities, so we can just add them. The expression simplifies to $1 - \mu/A$. This quantity is called p in the code. We can then find the next entry in the matrix, the transition probability between the states 'incorrect' and 'correct'. The only possibility here is mutating and becoming correct, which has a probability of μ/A . As a sanity check, adding this to the previous entry results in 1, as expected. The third entry in the matrix is the transition probability from 'correct' to 'incorrect'. Rather like the second entry, there is only one possibility, that it mutates and becomes incorrect. The probability of this is $\mu(1 - 1/A)$, which is called q in the code. Finally, the last entry of the transition matrix is the probability from 'correct' to 'correct'. Much like the first entry, there are two possibilities: either it stays the same or it mutates into the same character. The probability is then $1 - \mu + \mu/A$, which sums to 1 with the third entry.

Now we can move to the case of $L > 1$. All the characters are independent, so we can just take the tensor product of the separate 2x2 matrices for each character. However, there is a snag: this would give us a matrix of size 2^L . This is not what we want, because we want to consider only the probabilities of strings changing from Hamming distance L , $L - 1$, etc from the target all the way to Hamming distance 0. Thus, our state space is of size $(L + 1) \times (L + 1)$ and we want a matrix of that size. We have to take some of the 2^L states and lump them together. It is also useful now to introduce the concept of 'score', or L minus the Hamming distance. A string that is completely incorrect has a score of 0. Now, after the tensor product and lumping the new matrix elements are, with the indices starting from 0:

$$a_{jk} = \sum_i \binom{L-j}{k-j+i} (1-p)^{k-j+i} p^{L-k-i} \binom{j}{i} q^i (1-q)^{j-i}$$

where the binomial coefficients come from the number of times the fundamental units are repeated. This formula was guessed by essentially writing down the product $(1-p)^a p^b q^c (1-q)^d$ and then figuring out what the powers should be by looking at examples of the lumped matrices generated with sympy. The binomial coefficients become 0 when their arguments are negative in this convention.

We can now use what we have learned to do the full problem. This is where the selection comes in. We have to create another transition matrix.

For the maximum score of all of the offspring to be 0, all the offspring have to have a score of 0. So, for the first column, which quantifies the probabilities of transitions to a maximum score of 0, just take the probabilities of the first column of the previous matrix describing mutations to the N th power, since there are N offspring. For the second column, describing transitions to a maximum score of 1, we want all of the scores to be 0 or 1 but *not* all of them to be 0 so it's $(a_{00} + a_{01})^N - a_{00}^N$, where the a_{ij} are elements of the previous matrix. For the next one the scores should be either 0, 1, or 2, but not all 1 or 0 so it's $(a_{00} + a_{01} + a_{02})^N - (a_{00} + a_{01})^N$, and it continues like that, each time subtracting the cumulative sum of previous entries in the row. The a in each row sum to 1, so the rows of this new transition matrix also sum to 1, and this is still a valid transition matrix.

We can now use some results from Markov chain theory to find the distributions we are after. We have an absorbing chain (we can force it to be by making the entry in the lower right corner 1), so we can now use the 'discrete phase-type' distribution to calculate the time to absorption, or equivalently the time to completion of the process. The CDF as a function of k is³ $1 - \tau T^k \mathbf{1}$, where τ is a vector of the right dimensions that has first entry 1 and the other entries 0 (the initial state of the chain), and $\mathbf{1}$ is a vector of all 1s. T is the $L \times L$ square block of the transition matrix that does not include the 1 in the lower right corner. k is the number of steps. With the same symbols, the PDF is $\tau T^{k-1} T_0$, where $T_0 = (I - T) \mathbf{1}$. The mean is $\tau (I - T)^{-1} \mathbf{1}$.

³Theorem 2.4.1 of Latouche and Ramaswami, Introduction to Matrix-Analytic Methods in Stochastic Modeling (1999)